

Front-End

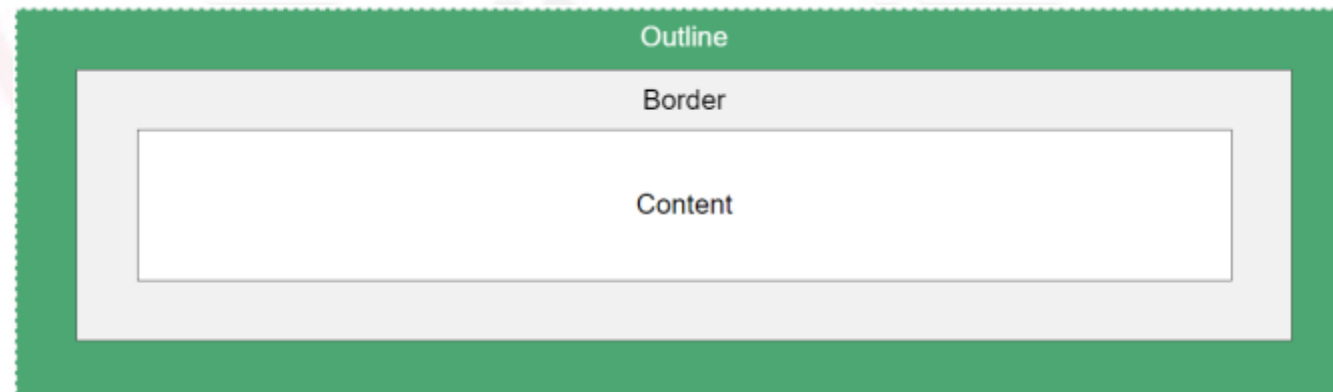
HTML CSS

Some CSS Tags

Outline

An outline is drawn **outside** an element's border to make it stand out.

- Unlike borders, outlines don't affect the element's size.
- They may overlap other content.



Example - outline-style: dotted;

```
<style>  
  div {  
    border: 1px solid red;  
    padding: 15px;  
    outline-style: dotted;  
  }  
</style>
```

```
<body>
```

```
  <div>This div element has a border and an outline.</div>
```

```
</body>
```

Example - outline-color: yellowgreen;

```
<style>
  div {
    border: 1px solid red;
    padding: 15px;
    outline-style: dotted;
    outline-color: yellowgreen;
  }
</style>
```

```
<body>
```

```
  <div>This div element has a border and an outline.</div>
```

```
</body>
```

Example - outline-width: 10px;

```
<style>
  div {
    border: 1px solid red;
    padding: 15px;
    outline-style: dashed;
    outline-color: yellowgreen;
    outline-width: 10px;
  }
</style>
```

```
<body>
```

```
  <div>This div element has a border and an outline.</div>
```

```
</body>
```

Shorthand property - Outline

Shorthand Property

Shorthand lets you set multiple related properties in one line.

Example (outline width, style, color):

```
p {  
    outline: 3px dotted red;  
}
```

Example

```
<style>
```

```
  /* shorthand property for outline */
```

```
  div {
```

```
    outline: 2px solid red;
```

```
    padding: 10px;
```

```
  }
```

```
</style>
```

```
<body>
```

```
  <div>This div element has a border and an outline.</div>
```

```
</body>
```

Example - Outline-offset

Outline-offset

Defines the space between an element's border and its outline.

```
<style>
  div {
    margin-top: 30px;
    padding: 15px;
    border: 5px solid red;
    outline: 5px solid yellowgreen;
    outline-offset: 15px;
  }
</style>
```

```
<body>
  <div>This div element has a border and an outline.</div>
</body>
```


CSS Box Model

CSS Box Model

The CSS Box Model defines how elements are displayed as rectangular boxes with four parts:

- **Content** – The actual content
- **Padding** – Space between content and border
- **Border** – Surrounds padding and content
- **Margin** – Space outside the border

It's key for controlling layout, spacing, and positioning into browsers.

Example

```
<style>
  div {
    width: 320px;
    height: 50px;
    padding: 10px;
    border: 5px solid gray;
    margin: 0;
  }
</style>
```

```
<body>
```

```
<h2>Calculate the total width of element</h2>
```

```

```

```
<div>The picture above is 350px wide. The total width of this element is also 350px.</div>
```

```
</body>
```

320px (width of content area)
+ 20px (left padding + right padding)
+ 10px (left border + right border)
= 350px (total width)

50px (height of content area)
+ 20px (top padding + bottom padding)
+ 10px (top border + bottom border)
= 80px (total height)

Felix -ITs



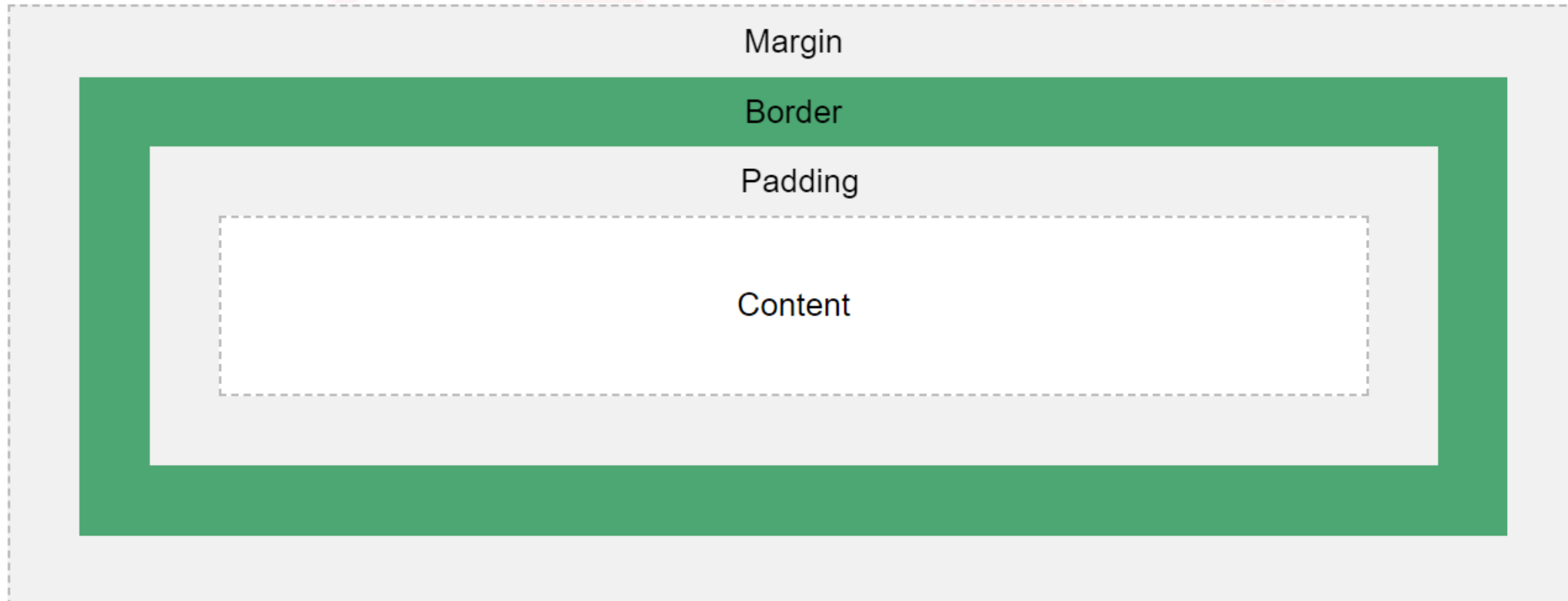
The total width of an element should be calculated like this.

Total element width = width + left padding + right padding + left border + right border

The total height of an element should be calculated like this

Total element height = height + top padding + bottom padding + top border + bottom border

☞ *Note:* Margins add extra space **outside** the box but are **not part of its actual size**.



Example 2

```
<style>
  .box {
    width: 200px;    /* Set width of content area */
    height: 100px;   /* Set height of content area */
    padding: 20px;   /* Add padding around content */
    border: 2px solid black; /* Add border around padding */
    margin: 30px;    /* Add margin around border */
  }
</style>

<body>
  <div class="box">
    This is the content.
  </div>
</body>
```

Total width = width + 2 * padding + 2 * border width

Total height = height + 2 * padding + 2 * border width

Total width = 200px + 2 * 20px + 2 * 2px = 244px

Total height = 100px + 2 * 20px + 2 * 2px = 144px

Total size of the div element is 244px wide and 144px tall.

Selectors

Selectors are patterns used to target and style HTML elements.

Common Types:

- Element → by tag name
- Class → by .class
- ID → by #id
- Attribute → by [attr]
- Descendant → element inside another
- Child → direct child
- Adjacent Sibling → next sibling
- Pseudo-classes / Pseudo-elements → by state or position

Using selectors the right way helps designers pick the exact elements they want and style them to make web pages look neat and attractive.

Example

Element Selector: Selects elements by tag name

```
<style>
  /* Element Selector */
  h1 {
    color: blue;
  }
</style>

<body>
  <h1>Element Selector Example</h1>
</body>
```

Example

Class Selector: Selects elements by class attribute

```
<style>
  /* Class Selector */
  .p1 {
    font-weight: bold;
  }
</style>
```

```
<body>
  <p class="p1">Class Selector Example</p>
</body>
```

Example

ID Selector: Selects a single element by its unique ID attribute

```
<style>
  /* ID Selector */
  #unique {
    text-decoration: underline;
  }
</style>
</head>
<body>
  <p id="unique">ID Selector Example</p>
</body>
```

Example

Attribute Selector: Selects elements by attributes

```
<head>
  <style>
    /* Attribute Selector */
    input[type="submit"] {
      background-color: red;
      color: white;
      padding: 5px 10px;
    }
  </style>

  <body>
    <input type="submit" value="Submit">
  </body>
```

Example

Descendant Selector: Selects elements that are descendants of another element

```
<style>
  /* Descendant Selector */
  div p {
    font-style: italic;
    text-align: center;
    font-size: 25px;
  }
</style>
```

```
<body>
  <div>
    <p>Descendant Selector Example</p>
  </div>
</body>
```

Example

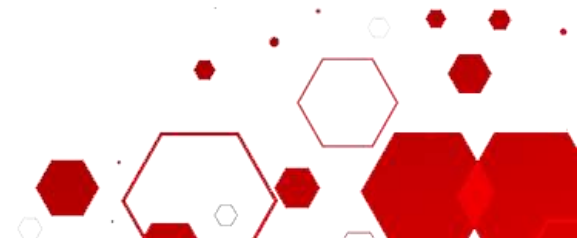
Child Selector: Selects elements that are direct children of another element

Selects and styles `<li>` elements that are direct children of `<ul>` with removed list-style.

```
<style>
  /* Child Selector */
  ul>li {
    list-style: none;
  }
</style>
```

```
<body>
  <ul>
    <li>Child Selector Example</li>
    <li>Child Selector Example</li>
    <li>Child Selector Example</li>
  </ul>
</body>
```

Felix -ITs



Example - Child Selector

```
<style>
  div>p {
    background-color: yellow;
  }
</style>

<body>

  <div>
    <p>Paragraph 1 in the div.</p>
    <p>Paragraph 2 in the div.</p>
    <section>
      <!-- not Child but Descendant -->
      <p>Paragraph 3 in the div (inside a section element).</p>
    </section>
    <p>Paragraph 4 in the div.</p>
  </div>

  <p>Paragraph 5. Not in a div.</p>
  <p>Paragraph 6. Not in a div.</p>

</body>
```

Example

The adjacent sibling selector is used to select an element that is directly after another specific element.

Sibling elements must have the same parent element and adjacent means immediately following.

```
<style>
  /* Adjacent Sibling Selector */
  h2+p {
    font-size: 18px;
    color: red;
    text-align: center;
  }
</style>

<body>
  <h2>Adjacent Sibling Selector Example</h2>
  <p>This paragraph is immediately preceded by an h2.</p>
</body>
```


Example - Adjacent Sibling Selector

```
<head>
  <style>
    div+p {
      background-color: yellow;
    }
  </style>
</head>

<body>

  <div>
    <p>Paragraph 1 in the div.</p>
    <p>Paragraph 2 in the div.</p>
  </div>

  <p>Paragraph 3. After a div.</p>
  <p>Paragraph 4. After a div.</p>

  <div>
    <p>Paragraph 5 in the div.</p>
    <p>Paragraph 6 in the div.</p>
  </div>

  <p>Paragraph 7. After a div.</p>
  <p>Paragraph 8. After a div.</p>

</body>
```

Example - Pseudo-elements

Pseudo-elements let you style parts of an element that are not part of the actual HTML markup.

You can use them to style elements like the first letter or line of text, or to add content before or after an element.

Syntax

```
selector::pseudo-element {  
  property: value;  
}
```

```
<style>
  p::first-line {
    color: red;
    background-color: antiquewhite;
    font-variant: small-caps;
    font-size: 20px;
  }
</style>
```

```
<body>
```

```
  <p> You can use the ::first-line pseudo-element to add a special effect to
the first line of a text. Some more text. And even more, and more, and more,
and more, and more, and more, and more, and more, and more, and more,
and more.
```

```
  </p>
```

```
</body>
```



!important Rule

!important rule in CSS is used to add more importance to a property/value than normal.

If you use the **!important** rule, it will override ALL previous styling rules for that specific property on that element.

```
<style>
  p {
    color: blue;
  }

  p {
    color: red !important;
  }
</style>
```

```
<body>
  <p>Hello from P tag</p>
</body>
```

Example 2

```
<style>
  #p1 {
    background-color: blue;
  }

  .p2 {
    background-color: gray;
  }

  p {
    background-color: red !important;
  }
</style>
</head>

<body>

  <p id="p1">This is some text in a paragraph.</p>

  <p class="p2">This is some text in a paragraph.</p>

  <p>This is some text in a paragraph.</p>

</body>
```

Responsive web design

Responsive web design ensures websites look and work well on all devices, from desktops to smartphones.

Key Practices:

- Viewport Meta Tag – Control layout on mobile browsers
- Fluid Layouts – Use relative units (%) instead of fixed px
- Media Queries – Apply styles for different screen sizes
- Flexible Images & Media – Use max-width: 100% for scaling

Responsive Typography – Use em, rem, or vw units

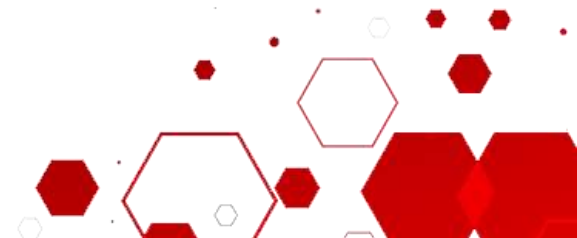
Mobile-First Approach – Start with small screens, scale up

Flexbox & Grid – Create flexible, adaptive layouts

Testing – Check across devices & browsers

Performance – Optimize images, CSS, and scripts for speed

☞ This approach guarantees a smooth user experience everywhere.



Media Query

Media queries let you apply CSS styles based on device features like screen size, orientation, or resolution.

They are the backbone of **responsive web design**, making websites adapt to different devices.

Syntax:

```
@media media_type and (media_feature) {  
  /* CSS styles */  
}
```

Common Media Types:

- **all** → all devices
- **screen** → desktops, tablets, phones
- **print** → when printing
- **speech** → screen readers

Common Media Features:

- **width / height** → viewport size
- **orientation** → landscape / portrait
- **aspect-ratio** → width-to-height ratio
- **resolution** → pixel density (dpi, ppi)

```
/* For small screens */  
@media screen and (max-width: 600px) {  
  body { font-size: 14px; }  
}
```

```
/* For high-resolution screens */  
@media screen and (min-resolution: 300dpi) {  
  img { max-width: 100%; }  
}
```

```
/* For printing */  
@media print {  
  body { color: black; background: none; }  
}
```

☞ In simple words: *Media queries tell the browser —*

“if the device looks like this, apply these styles.”

Example

```
<style>
  /* Default background color */
  body {
    background-color: #ffffff;
  }

  /* Media query for screens with a maximum width of 900 pixels */
  @media screen and (max-width: 900px) {
    body {
      background-color: #ff0000;
    }
  }

  /* Styles for content */
  .content {
    max-width: 800px;
    margin: 0 auto;
    padding: 20px;
    text-align: center;
    font-family: Arial, sans-serif;
  }
</style>
```

```
<body>
  <div class="content">
    <h1>This is main heading</h1>
    <p>This background color will change on smaller screens.</p>
  </div>
</body>
```

Felix -ITs

Example 2

```
<style>
```

```
/* Default background color */
```

```
body {
```

```
    background-color: #ffffff;
```

```
}
```

```
h1 {
```

```
    background-color: black;
```

```
    color: yellow;
```

```
}
```

```
/* Media query for screens with a maximum width of 900 pixels */
```

```
@media screen and (max-width: 900px) {
```

```
    body {
```

```
        background-color: #076576;
```

```
    }
```

```
    h1 {
```

```
        background-color: red;
```

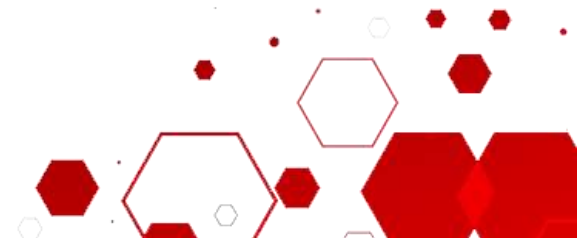
```
        color: white;
```

```
        padding: 8px;
```

```
    }
```

```
}
```

```
</style>
```



```
<body>
  <div>
    <h1>Media Query</h1>
  </div>
</body>
```

Felix -ITs

Example 3

<style>

```
/* Default background color */
body {
    background-color: #ffffff;
}

h1 {
    background-color: black;
    color: yellow;
}

/* Media query for screens with a maximum width of 900 pixels */
@media screen and (max-width: 900px) {
    body {
        background-color: #076576;
    }

    h1 {
        background-color: red;
        color: white;
        padding: 8px;
    }
}
```



```
/* Media query for screens with a maximum width of 600 pixels */  
@media screen and (max-width: 600px) {  
  body {  
    background-color: #fcfcfc;  
  }  
  
  h1 {  
    display: none;  
  }  
}  
</style>
```

```
<body>  
  <div>  
    <h1>Media Query</h1>  
  </div>  
</body>
```

Background Attachment

- Fixed background image – right bottom

```
<style>
  body {
    background-image: url("img.png");
    background-repeat: no-repeat;
    background-position: right bottom;
    background-attachment: fixed;
  }
</style>
```

To center an image horizontally and/or vertically

```
<style>
  div {
    display: flex;
    align-items: center;
    justify-content: center;
  }
  img {
    /* max-width: 100%;
    margin: auto; */
  }
</style>
</head>
<body>
  <div>
    
  </div>
</body>
```

Centering the content – Div Section

```
<style>
  #divCenter {
    display: flex;
    justify-content: center;
    align-items: center;
  }

  #div1 {
    border: 1px solid black;
    width: 500px;
    padding: 20px;
    text-align: center;
  }
</style>
```

```
<body>

  <div id="divCenter">
    <div id="div1">
      <h1>Hello world</h1>
      <p>This is paragraph</p>
      <button>Click here</button>
    </div>
  </div>

</body>
```

How to make image responsive in CSS

```
<style>
  img {
    max-width: 100%;
    height: auto;
  }
</style>
</head>

<body>

</body>
```

How to Make YouTube Video Responsive

```
<style>
  iframe {
    width: 700px;
    height: 500px;
    max-width: 100%;
    margin: auto;
  }
</style>
</head>

<body>

  <iframe src="https://www.youtube.com/embed/qz0aGYrrlhU?si=ID3X-itlWXVs1Eh" title="YouTube video player"
    frameborder="0"
    allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture; web-share"
    referrerpolicy="strict-origin-when-cross-origin" allowfullscreen></iframe>

</body>
```

Practice for Lecture 9 - 1

- ✓ **Watch the video carefully and write the code shown in it.**
-  **Upload your code to GitHub** after completing the practice.
- ? Don't worry if you don't fully understand some lines of code — just follow along with the video for now.
- Understanding will come with practice! 💪

Video Link

https://drive.google.com/drive/folders/1H_ddiuyean2lCslyuXJu_xERs1XcvpaH?usp=sharing

Practice for Lecture 9 - 2

 **Write answers to the following questions in your notebook in your own words.**

- What is an outline in CSS and what are its uses?
- What are selectors in CSS and what are their types?
- What is the universal selector in CSS?
- What are pseudo-elements and pseudo-classes in CSS?
- What is !important in CSS?
- What is responsive web design and what steps are taken to make a website responsive?
- What is a media query in CSS?
- How can you center an image using CSS?
- How can you make images and videos responsive in CSS?

 Practice for Lecture 9 - 3

 Record Videos in English on the Following Topics

- 1. What would you do if your teammate is not contributing?*
- 2. What would you do if you disagree with your manager?*
- 3. How do you handle criticism from peers or seniors?*
- 4. What will you do if someone takes credit for your work?*

 Speak clearly and confidently in English.

This will help improve your communication and interview skills!



Thank You



Felix -ITs

Felix -ITs

